Colecciones en C#

Ing. Andrés Eduardo Paternina Ariza Esp.

Universidad del Magdalena

apaternina@unimagdalena.edu.co

Ejercicio – Notas de tres parciales

Con las notas de tres parciales, calcula el promedio y muestra “Aprobado” si es mayor o igual a 3.0, “Recuperación” entre 2.0 y 2.9, y “Reprobado” si es menor a 2.0. Además, indica cuál de las tres notas fue la más alta.

ENTRADA
Tres notas fijas en el código.

PROCESO
Promedio y comparación de las tres notas.

SALIDA
Promedio, estado y nota más alta.

Solución en C#: notas de tres parciales

```
double n1 = 3.5, n2 = 2.8, n3 = 4.1;
double promedio = (n1 + n2 + n3) / 3;

if (promedio >= 3.0)
{
    Console.WriteLine("Aprobado");
}
else if (promedio >= 2.0)
{
    Console.WriteLine("Recuperación");
}
else
{
    Console.WriteLine("Reprobado");
}

double mayor = n1;
if (n2 > mayor) { mayor = n2; }
if (n3 > mayor) { mayor = n3; }

Console.WriteLine($"Promedio: {promedio:F2}");
Console.WriteLine($"Nota más alta: {mayor}");
```

Colecciones en C#

ARRAY

Tamaño fijo

```
int[] precios =
  new int[] {10, 20, 30};

precios[0] = 15;
int n = precios.Length;
```

LIST<T>

Tamaño dinámico

```
List<string> cats =
  new List<string>();

cats.Add("libros");
cats.Add("ropa");
int n = cats.Count;
```

DICTIONARY<K,V>

Pares clave → valor

```
var stock =
  new Dictionary<string,int>();

stock["libros"] = 42;
int q = stock["libros"];
```

Ejercicio – Ventas de la semana

Una empresa desea registrar las ventas de los 7 días de la semana. Almacena las ventas diarias en un arreglo y muestra el total vendido y el promedio.

ENTRADA
Las 7 ventas diarias, una por día.

PROCESO
Recorrer el arreglo, acumular la suma y dividir por Length.

SALIDA
Total vendido y promedio diario.

Solución – Ventas de la semana

SOLUCIÓN EN C#

```
double[] ventas = new double[7];
string[] dias = { "Lun", "Mar", "Mie", "Jue", "Vie", "Sab", "Dom" };

for (int i = 0; i < ventas.Length; i++)
{
    Console.Write($"Ventas de {dias[i]}: ");
    ventas[i] = double.Parse(Console.ReadLine());
}

double total = 0;
foreach (double v in ventas)
{
    total += v;
}
double promedio = total / ventas.Length;

Console.WriteLine($"Total vendido: {total}");
Console.WriteLine($"Promedio diario: {promedio:F2}");
```

Ejercicio – Carrito de compras con List

Una tienda registra los productos que el cliente agrega al carrito. No se sabe cuántos comprará: agrégalos a una lista y al final muestra todos los registrados.

ENTRADA
Nombres de productos, cantidad no conocida.

PROCESO
carrito.Add() dentro de un while hasta escribir 'fin'.

SALIDA
Cuántos productos y el listado completo.

Solución – Carrito de compras

SOLUCIÓN EN C#

```
List<string> carrito = new List<string>();
string producto;
bool seguir = true;
while (seguir)
{
    Console.Write("Producto (o 'fin' para terminar): ");
    producto = Console.ReadLine();
    if (producto.ToLower() != "fin")
    {
        carrito.Add(producto);
    }
    else
    {
        seguir = false;
    }
}
Console.WriteLine($"Productos: {carrito.Count}");
foreach (string p in carrito)
{
    Console.WriteLine($"- {p}");
}
```

Ejercicio – Notas de estudiantes

Un profesor desea registrar las notas de sus estudiantes. Almacena en un Array las notas de 5 estudiantes y muestra cuál fue la nota más alta y cuál la más baja.

ENTRADA
Las notas de 5 estudiantes.

PROCESO
Recorrer el arreglo comparando cada nota.

SALIDA
La nota más alta y la más baja.

Solución – Notas de estudiantes

SOLUCIÓN EN C#

```
double[] notas = new double[5];
for (int i = 0; i < notas.Length; i++)
{
    Console.Write($"Nota del estudiante {i + 1}: ");
    notas[i] = double.Parse(Console.ReadLine());
}
double mayor = notas[0];
double menor = notas[0];
foreach (double n in notas)
{
    if (n > mayor)
    {
        mayor = n;
    }
    if (n < menor)
    {
        menor = n;
    }
}
Console.WriteLine($"Nota más alta: {mayor}");
Console.WriteLine($"Nota más baja: {menor}");
```

Que tendríamos que cambiar para recibir N notas?

LINQ en C#

QUE ES?

Lenguaje de consultas integrado

Language Integrated Query: permite consultar colecciones de forma declarativa.
Requiere: `using System.Linq;`

OPERADORES COMUNES

Where, Select, OrderBy, Sum...

```
Where(x => x > 10)
Select(x => x * 2)
OrderBy(x => x)
Sum(), Count()
Average(), Any()
First(), ToList()
```

EJEMPLO

Sintaxis fluida con List

```
var nums =
  new List<int>
    {5, 12, 8, 20};

var grandes = nums
  .Where(n => n > 10)
  .ToList();
// [12, 20]
```

Ejercicio – Precios con LINQ

Una tienda tiene una lista con los precios de varios productos. Usa LINQ para mostrar solo los productos con precio mayor a $50.000, cuántos cumplen la condición y ordénalos de menor a mayor precio.

ENTRADA
Una lista con los precios de varios productos.

PROCESO
Where para filtrar y OrderBy para ordenar.

SALIDA
Cuántos superan $50.000 y la lista ordenada.

Solución – Precios con LINQ

SOLUCIÓN EN C#

```
// requiere: using System.Linq;
List<double> precios = new List<double>
{ 35000, 72000, 51000, 18000, 125000, 50000 };

List<double> caros = precios
    .Where(p => p > 50000)
    .OrderBy(p => p)
    .ToList();

Console.WriteLine($"Productos sobre $50.000: {caros.Count}");
foreach (double p in caros)
{
    Console.WriteLine($"- {p:N0}");
}
```

Ejercicio – Ventas de la semana

Una empresa desea registrar las ventas de los 7 días de la semana. Almacena las ventas diarias en un arreglo y muestra el total vendido y el promedio.

ENTRADA
Las 7 ventas diarias, una por día.

PROCESO
Recorrer el arreglo, acumular la suma y dividir por Length.

SALIDA
Total vendido y promedio diario.

¿Qué pasa con los string?

Declara una variable string y escribe un punto (.) después del nombre de la variable.

PRUÉBALO EN VISUAL STUDIO

```
string nombre = "Andrés";
nombre.
```

Tarea: List y LINQ

NUMEROS

1. Crea una List<int> con: 4, 9, 2, 15, 7, 22, 11. Imprime solo los mayores a 10.
2. Calcula la suma y el promedio de los numeros pares de la lista anterior.
3. Dada un List<string> con nombres, devuelve solo los que empiezan con 'A'.